Cheating in Poker Challenge: Final Report

1st Andy Zheng
Department of EECS
Massachusetts Institute of Technology
akzheng@mit.edu

2nd Christopher Liem
Department of EECS
Massachusetts Institute of Technology
cliem@mit.edu

***Abstract*—In our variant of 4 player Leduc poker, teammates are allowed to "cheat" by using a correlated strategy based on global signal. Once a round start, they are not allowed to communicate in any form. To find an effective equilibrium in this setting, we explored three approaches to this poker challenge. First, we implemented a heuristic-based method using expected value calculations across scenarios, which proved highly exploitable. Second, we developed a Team-Belief DAG solver based on past work, which represents team coordination as a directed acyclic graph of belief states and solves for correlated equilibrium strategies using CFR-based methods. While TB-DAG achieved moderate exploitability, its head-to-head performance was limited by the small number of correlated strategies we could train in the available time. Finally, we implemented the Team-PSRO framework, an iterative method that builds a diverse population of strategies by repeatdely computing best responses. Despite higher exploitability scores compared to other teams, Team-PSRO performed surprisingly well head-to-head, suggesting that strategy diversity may play a crucial factor in practice.**

I. Introduction

Cooperation in adversarial games remains to be a fundamental challenge in game theory today: how should teammates coordinate their strategies when they cannot communicate during play? When team members can communicate privately during gameplay, they effectively function as a single player with perfect recall. However, when in-game communication is prohibited, the asymmetry in information between team members prevents standard approaches from working, making optimal team strategy computation NP-hard in general.

This report addresses a 4-player variant of Leduc Hold'em where Players 1 and 3 form one team against Players 2 and 4. The game uses a six-card deck with two each of Jack,Queen,King, and proceed through two betting rounds with fixed raise amounts. The key feature distinguishing this game with standard Leduc Poker is that teammates are permitted to "cheat" in a limited manner. Before each game, team members may jointly sample from a shared global signal, enabling coordination on a correlated strategy. Once play begins for a game, no further communication is allowed. This setting corresponds to the Team-Maxmin Equilibrium with Correlation (TMECor) solution concepts, which seeks the team strategy maximizing expected utility against a best-responding opponent.

Throughout the challenge, we explored three approaches: a heuristic strategy large based on card signalling and best responses in terms of pure expected values in numerous scenarios, a Team Belief DAG solver applying CFR-based methods on a belief-state representation TB-DAG, and Team Psro, an iterative method using cooperative reinforcement learning. The following sections detail our methodology, experimental results and analysis.

II. Heuristic-Based Approach

Before exploring algorithmic solutions, we first developed several heuristic strategies to build intuition about the game dynamics through some playing of Leduc poker. One of our first ideas was to use the first round for the teammates to signal each other which cards they have, as the community card has not been revealed in that stage and it's hard to analyze the relative strength of the card you are holding. Then we performed some expected valid analysis to evaluate the best action for the second player to take based on the first player's signal.

Our primary signaling scheme encodes the first teammate's private card through their initial betting action, where the correspondence is listed below:

- Jack: Fold
- Queen: Check/Call
- King: Raise

This order is based on the card strength order of Jack $<$ Queen $<$ King. With this signal from the first teammate, then the second teammate (Player 3 or 4) can then decode this signal and combine it with their own private card to make more informed decisions. For example, if Player 1 signals that they have a Jack and Player 3 also holds a Jack, they know with certainty that they would no longer be able to win in this round, as the community card won't be a Jack and the opponents are certain to hold no Jack. Thus, Player 3 can deduce that folding would be the optimal action just based on Player 1's signaling. This shows that signaling can significantly affect optimal play. Our list of the "optimal" decision that the second player can make in different scenarios is listed below:

- Both teammates have Jacks: Fold immediately as there is no chance of winning unless the opponent folds, which is unlikely as they must hold Queens and Kings.

- First teammate signaled the same card as the second Teammate (QQ or KK): Play cautiously. Only one teammate can form a pair with the community card, and the opponents may hold the remaining copy.
- First teammate signaled differently as the second teammate: Play more aggressively. The probability that at least one teammate pairs with the community card is higher, and we have better coverage of the card space.

Since our 4-player Leduc poker has a relatively small deck, it was feasible to conduct expected value analysis and determine a "good" ration to concretely define what playing cautiously or aggressive means. As a result, we were able to submit a hard-coded scheme based on the ideas given above.

However, this hard coded scheme faces several practical limitations. The signaling conventions are transparent to opponents who can exploit predictable behavior. Furthermore, the signals carry inherent costs. For example, the first player folding with a Jack is not necessarily always the best strategy, as it may very well be possible that the community card later revealed is Jack. Due to these limitations, it was not a surpise that our heuristic approach proved highly exploitable in practice, which we will touch upon more in the results section. Nevertheless, developing these heuristics was still a valuable exercise, as it deepened our understanding of the game's information structure and the challenges of implicit team coordination. This motivated us to explore more principled algorithmic approaches capable of discovering less predictable and less exploitable coordination strategies.

III. Team-Belief DAG Based Approach

Interestingly, Team Belief DAG was the last approach our team explored. Our initial focus was on Team-PSRO, whose strong head-to-head performance gave us confidence despite its relatively high exploitability. However, as the challenge progressed, we observed that teams with more theoretically grounded, low-exploitability strategies were gaining an edge. This prompted us to revisit the literature and attempted to implement the Team Belief DAG framework.

A. Background on Team-Belief DAG

The Team Belief DAG framework, introduced by Zhang et al.[1], constructs a strategically equivalent perfect-information decision problem from the original team game. The key insight is that while individual team members have private information, the team can reason about join belief states, which are the set of possible game states consistent with the team's collective observations and actions.

The main tool for converting the original team game to a strategically equivalent perfect-information decision problem from the original team game is TB-DAG. The TB-DAG is a directed acyclic graph where each node corresponds to a belief state. The root is initialized with the game's root node. At each decision node, the team selects a perscription, which is a joint action specifying one action for each information set intersecting the current belief. At inactive nodes, where nature or opponents act, the DAG branches according to public observations that the team can commonly distinguish. In the paper, the authors proved that this construction is strategically equivalent to the original team game. In fact, the set of realization-form strategies in the DAG exactly corresponds to the set of correlated team strategies.

Once converted to a perfect-information DAG, standard regret minimization techniques become applicable. Classis techniques such as Counterfactual Regret Minimization (CFR) and its variants can be run directly on the DAG with some modifications, with regret bounded by $O(N\sqrt{T})$ after T iterations. Using these classic techniques, one can find an average startegy that converges to a TMECor.

B. Application to Team Leduc Poker

We implemented the Team Belief DAG framework for our setting in python, mainly following Algorithms 1 and 2 from Zhang et al.[1] . Our implementation consisted of code handling game tree parsing, TB-DAG construction, CFR solving, and strategy extraction.

To construct the corresponding TB-DAG structure for each of the teams, we followed Algorithm 1 in the original paper by recursively building belief nodes. A key step is computing public observations, which is the information commonly known to all team members. We implemented this using a connectivity graph where histories are connected if they belong the same information set for any team member. Public observation correspond to connected components of this graph restricted to the current belief state, computed by union-find for better efficiency.

For each belief node, we enumerate all prescriptions. The number of prescriptions can grow combinatorially with the number of overlapping ifnormation sets, but the structure of Leduc Poker keeps this manageable. After applying a prescription, we advance through opponent and chance nodes to compute the resulting child beliefs.

After constructing the corresponding TB-DAG structure for the two teams, we followed Algorithm 2 of the original paper to run a CFR+ based algorithm on the TB-DAG structure. Then we would let the two team's to self-play against each other to produce an adversarily robust strategy that is closer to true equilibrium. The specific process of self-play is described below:

- Both teams would compute their current strategies
- Then both teams would traverse the TB-DAG structures to compute terminal utilities under both strategies
- Then both teams' regrets will be updated for the next round

Speed and efficiency has been a huge technical challenge in this step. To reduce training time, we implemented a

UtilityComputer class to that precomputes lookup tables for efficient utility calculation, mapping histories to beliefs and caching path information.

With our implementation so far, we are essentially only outputting one optimal solution for both teams. However, the Poker challenge allows teams to correlate by a shared signal selecting among multiple strategies. To support this feature, we extended our solver to train multiple independent strategy profiles simultaneously. We maintain separate CFR+ solvers for each strategy index and learn the meta-strategy probabilities by an outer loop of regret matching.

Due to time restrictions, we are not entirely confident our implementation is fully correct. It was difficult to fully validate our code and check that it was correctly build the TB-DAG structure for our game tree in time. There is some evidence that our implementation is mishandling certain public observation computations or belief advancements, as our result after training a thousand iterations didn't perform as well as other teams who seemed to have implemented TB-DAG correctly.

Another potential concern is that our multi-strategy extension may not produce meaningfully diverse strategies. Training multiple independent CFR+ solvers initialized identically may simply converge to the same equilibrium. Without explicit diversity incentives or different initializations, there are no mechanism pushing the strategies apart. One future direction is definitely investing more time to concretely think of creative solutions to take in advantage of the fact that teams can use signals to use multiple correlated strategies.

IV. Team PSRO Based Approach

The main approach we took for this challenge was to implement Team Policy-Spaced Response Oracles (Team PSRO) [2]. Team-PSRO is a population-based equilibrium approximation algorithm for cooperative-adversarial games that targets the computation of TMECor. Rather than optimizing over the full joint policy space, Team-PSRO maintains restricted populations of team strategies and formulates a meta-game over these populations. At each iteration, equilibrium mixtures are computed in the meta-game, and the new team strategies are added using best-response oracles to the opponent's meta-strategy. This enables the scalable approximation of TMECor in large imperfect-information team games.

A. Game Representation

Before implementing Team-PSRO, we first construct an explicit representation of the Team Leduc Hold'em game. The game is represented as an extensive-form tree consisting of chance nodes, player decision nodes, and terminal utility nodes. Chance nodes encode the initial private card deal and subsequent public card reveals, with transition probabilities directly specified by the game definition and independent of player actions. Player nodes correspond to decision points for a single acting player and are labeled with the set of legal actions available at that state. Terminal nodes assign utilities to each player, and team utilities are obtained by aggregating the payoffs of the players belonging to the same team. In particular, the payoff for Team 13 at a terminal node is u_1+u_3, while the payoff for Team 24 is $u_2 + u_4$.

To model a player's observation of the game, information sets (infosets) are used. For each infoset, a player observes their own private card and the full public history of actions and chance events prior to the decision node, while the private cards of the other players are masked. To ensure consistency between the extensive-form game and the provided infoset specifications, we define a deterministic mapping from full game histories to player observations by masking unobserved private information and preserving all public transitions. This mapping allows every player decision node in the game tree to be associated with a unique infoset index, enabling direct lookup of behavioral strategies during evaluation.

Player strategies are represented as behavioral policies defined over information sets. For each player, a strategy assigns a probability distribution over the legal actions at every infoset. Illegal actions are explicitly masked, and probabilities are renormalized to ensure valid distributions. A team strategy is defined as a collection of independent behavioral strategies for the players on that team, with no additional coordination at the policy level. Thus, while teammates may coordinate on a joint strategy prior to the game, they do not exchange information or adapt their strategies during gameplay.

Given a joint strategy profile, expected utilities are computed by recursively traversing the extensive-form game tree. At terminal nodes, the expected utility for Team 13 is defined as the sum of utilities for players 1 and 3. At chance nodes, values are computed as expectations over successor states weighted by the corresponding transition probabilities. At player decision nodes, successor values are weighted by the acting player's behavioral strategy at the associated infoset. To improve efficiency, intermediate values are memoized so that each game history is evaluated at most once.

B. Meta-Game Construction and Solution

Team-PSRO operates by maintaining finite populations of team strategies for each team and defining a restricted normal-form meta-game over these populations. Each pure action in the meta-game corresponds to a complete team strategy, defined as a tuple of behavioral strategies for the players on that team. Let

$$\Pi_{13} = \{\pi_{13}^1, \ldots, \pi_{13}^m\} \quad \text{and} \quad \Pi_{24} = \{\pi_{24}^1, \ldots, \pi_{24}^n\}$$

denote the current populations of team strategies for Team 13 and Team 24, respectively. Interactions between these populations define a zero-sum payoff matrix $A \in \mathbb{R}^{m \times n}$, where each entry

$$A_{ij} = U_{13}(\pi_{13}^i, \pi_{24}^j)$$

is computed through exact evaluation of the underlying extensive-form game.

To compute equilibrium mixtures over the restricted strategy sets, we solve the induced meta-game using fictitious play. Let $\sigma_{13} \in \Delta(\Pi_{13})$ and $\sigma_{24} \in \Delta(\Pi_{24})$ denote mixed strategies over the populations. Fictitious play iteratively updates empirical distributions by computing best responses to the opponent's current mixture, yielding approximate equilibrium mixtures for the restricted meta-game. These meta-strategies define correlated distributions over team strategies and serve as the basis for population expansion. Intuitively, to solve for a team's meta strategy, fix the opponent's strategy and compute the best response strategy using the payoff matrix A. Then, keep track of how many times each strategy was played, normalizing the counts to obtain probability distributions. The algorithm is outlined in 1.

Algorithm 1 Fictitious Play

Require: Zero-sum payoff matrix $A \in \mathbb{R}^{m \times n}$, number of iterations T
Ensure: Meta strategies $\sigma_{13} \in \Delta(\Pi_{13})$, $\sigma_{24} \in \Delta(\Pi_{24})$
Initialize row counts $c_{13} \leftarrow \mathbf{1}_m$
Initialize column counts $c_{24} \leftarrow \mathbf{1}_n$
$\sigma_{13} \leftarrow c_{13} / \sum_i c_{13}(i)$
$\sigma_{24} \leftarrow c_{24} / \sum_j c_{24}(j)$
for $t = 1$ to T **do**
    Team 13 best response:
    $i^* \leftarrow \arg\max_i (A\sigma_{24})_i$
    $c_{13}(i^*) \leftarrow c_{13}(i^*) + 1$
    Team 24 best response:
    $j^* \leftarrow \arg\min_j (\sigma_{13}^\intercal A)_j$
    $c_{24}(j^*) \leftarrow c_{24}(j^*) + 1$
    Update mixed strategies:
    $\sigma_{13} \leftarrow c_{13} / \sum_i c_{13}(i)$
    $\sigma_{24} \leftarrow c_{24} / \sum_j c_{24}(j)$
end for

C. Team-PSRO Population Expansion

To expand the populations of strategies, Team-PSRO computes approximate best responses to the opponent's meta-strategy using a public-belief Counterfactual Regret Minimization (CFR) oracle. Given a meta-strategy σ_{-T} for the opposing team, we construct a stationary opponent behavioral strategy by linearly combining their population strategies:

$$\overline{\pi}_{-T}(I) = \sum_j \sigma_{-T}(j)\, \pi^j_{-T}(I),$$

for every information set I. This yields a fixed opponent policy defined over all infosets.

Holding the opponent's policy fixed, we compute a best response for the acting team by running CFR for the two players on that team. CFR minimizes counterfactual regret at each information set by repeatedly updating regrets based on counterfactual action values and applying regret matching to derive behavioral strategies. Strategy averages are accumulated across iterations, and the resulting average strategies define an approximate best-response team strategy.

These newly generated strategies are appended to the corresponding team populations, and the meta-game is recomputed. This process is repeated for a fixed number of iterations. As the populations grow, the restricted meta-game becomes a progressively tighter approximation of the original game, and the resulting meta-strategies approach a Team Max-Min Equilibrium with Correlation (TMECor).

Unlike [2], which employs reinforcement-learning-based best-response oracles such as MAPPO, we use a public-belief CFR oracle to compute approximate best responses directly in the extensive-form game. This choice leverages the fully known, tabular structure of Team Leduc Hold'em, enabling exact counterfactual reasoning and variance-free evaluation while retaining the core population-based structure of Team-PSRO. The full algorithm is given in algorithm 2.

Algorithm 2 Team PSRO

Require: Initial team populations Π^1_{13}, Π^1_{24}, number of iterations K.
Ensure: Meta strategies σ_{13}, σ_{24}
for $t = 0, 1, \ldots, K-1$ **do**
    Meta-game construction:
    for $\pi^i_{13} \in \Pi^t_{13}$ **do**
        for $\pi^j_{24} \in \Pi^t_{24}$ **do**
            $A^t_{ij} \leftarrow U_{13}(\pi^i_{13}, \pi^j_{24})$
        end for
    end for
    Meta-game solution:
    Compute meta strategies

$$\sigma^t_{13} \in \Delta(\Pi^t_{13}), \quad \sigma^t_{24} \in \Delta(\Pi^t_{24})$$

    using fictitious play (Alg. 1) on payoff matrix A^t.
    Population expansion for Team 13:
    Construct opponent policy

$$\overline{\pi}_{24} \leftarrow \sum_{j=1}^{|\Pi^t_{24}|} \sigma^t_{24}(j)\pi^j_{24}$$

    $\pi^{\text{BR}}_{13} \leftarrow$ CFR-BR$(\overline{\pi}_{24})$
    $\Pi^{t+1}_{13} \leftarrow \Pi^t_{13} \cup \{\pi^{\text{BR}}_{13}\}$
    Population expansion for Team 24:
    Construct opponent policy

$$\overline{\pi}_{13} \leftarrow \sum_{i=1}^{|\Pi^t_{13}|} \sigma^t_{13}(i)\pi^i_{13}$$

    $\pi^{\text{BR}}_{24} \leftarrow$ CFR-BR$(\overline{\pi}_{13})$
    $\Pi^{t+1}_{24} \leftarrow \Pi^t_{24} \cup \{\pi^{\text{BR}}_{24}\}$
end for
Return final meta-strategies $\sigma^K_{13}, \sigma^K_{24}$

V. Results

A. Heuristics

The heuristic-based approach did not perform very well, even in the early stages of the competition. There are several

reasons for this: (1) not enough meta strategies could be produced and (2) it was unclear where the probability mass should be distributed. Since we had to hardcode all the probability distributions ourselves, it was nearly impossible to produce numerous meta strategies that were independent from one another. As a result, we ended up with high exploitability rates between 5-6, never going below that threshold. Moreover, we didn't want actions to be deterministic so we had to figure out how to distribute the probability mass ourselves. Without simulating gameplay, we were unable to identify which actions worked best in which scenario. This resulted in suboptimal strategies, which was evident in us losing against nearly every team in the early stages of the competition. This motivated us to lean towards learning-based approaches instead.

B. Team Belief DAG

For the results of our two variants of the Team-DAG approach, the exploitability of the meta-strategy produced by our code are shown by the charts below:

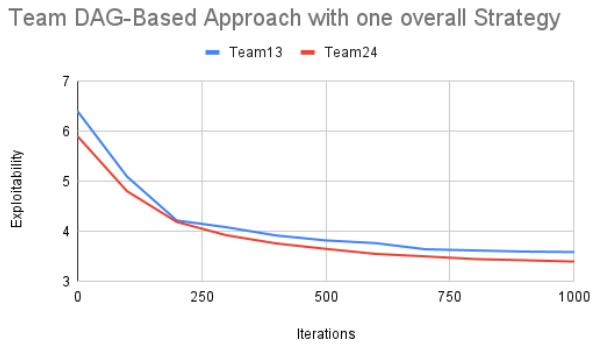


Fig. 1. Exploitability over iterations for one optimized strategy through TB-DAG Self-Play

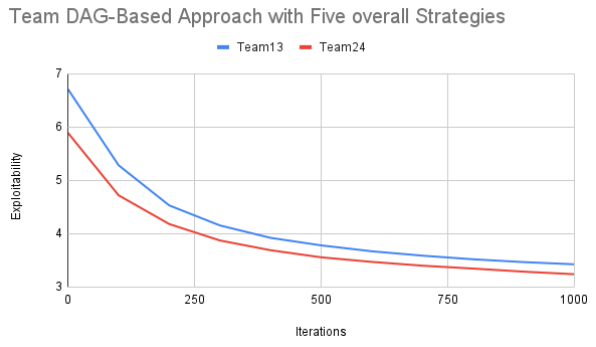


Fig. 2. Exploitability over iterations for five independently optimized strategy through TB-DAG Self-Play

For the Team Belief DAG approach, both the single strategy and five-strategy variants yielded similar exploitability scores over 1000 iterations. Exploitability decreased from approximately from 6.3 to 3.6 for Team 13 and from 6.1 to 3.4 for Team 24 under both settings. These results represent an improvement over both the uniform random strategy and out heuristic signaling approach.

However, there are several things concerns with out approach. The exploitability curves appear to plateau well above aero, rather than continuing to converge toward an equilirbium solution. A correctly implemented TB-DAG solver running CFR+ shoudl approach zero exploitability given sufficient iterations. The persistent gap suggest a poential implementation error in our DAG construction or utility computation. Additionally, our multi-strategy extension, which involves training five independent strategies via separate CFR+ runs on the TB-DAG, didn't meaningfully outperform the single strategy baseline. This indicates that simply running multiple independent solvers without more thought does not produce the strategic diversity needed to benefit from correlations. With more time, more principled apparoches to meta-strategies are required.

Compared to other teams in the competiton, our TB-DAG based strategy underperformed compared to other teams in terms of explotiability. Whiel insufficient training time is one possible explaination, past result indicate that 1000 iterations should yield meaningful convergence for games of this scale. The plateauing trend in our exploitability curves makes it difficult to attribute the gap solely to iteration count.

In head-to-head evaluation, our TB-DAG based strategy achieved positive utility against only 8 of the the 22 non-uniform team submissions. Despite the underwhelming performance, the exploration was valuable for understanding the TB-DAG framework and its implementation challenges. Given additional time, we would focus on debugging the DAG construciton, validating intermediate belief states, and developing more principled methods for learning diverse correlated strategies.

C. Team PSRO

Four sets of strategies for both teams were produced under Team PSRO with the following configurations:

1) 100 rounds of fictitious play (T=100), and 10 iterations of Team PSRO (K=10)
2) 200 rounds of fictitious play (T=200), and 30 iterations of Team PSRO (K=30)
3) 100 rounds of fictitious play (T=100), and 30 iterations of Team PSRO (K=30)
4) 500 rounds of fictitious play (T=500), and 50 iterations of Team PSRO (K=50)

Configuration (1) had a training time of approximately four hours, configuration (2) had a training time of around twenty hours, configuration (3) had a training time of around sixteen hours, and configuration (4) had a training time of around sixty hours. Since training took hours, it was impractical to test numerous strategies.

Moreover, configuration (2) resulted in exploitabilities [3.8, 3.3] for teams 13 and 24 respectively, configuration (3)

resulted in exploitabilities [3.6, 3.0], and configuration (4) resulted in exploitabilities [3.2, 2.7].

Interestingly, configuration (3) had lower exploitability than configuration (2) despite having 100 less rounds of fictitious play. We believe that the variance in exploitability of fictitious games results from overfitting to an incomplete population. In particular, in the early iterations of Team PSRO, we have meta-strategies that are incomplete and suboptimal since many possibilities are still unexplored. Thus, a bad approximation of the real game is computed, which allows fictitious play to heavily overweight bad strategies. As a result, a meta strategy that is optimal within the restricted game but highly exploitable once a new strategy is introduced is created, increasing our overall exploitability.

On the other hand, increasing the number of Team PSRO iterations strictly decreased exploitability, though we don't have many configurations to analyze. This is because as we increase the number of iterations, we are continuously producing more strategies to work with, increasing the variability in our play style and causing us to be less predictable.

However, our measured exploitability remained relatively high compared to other teams, typically ranging between 2.7 and 3.8 across configurations. Despite this, our agent outperformed more than half of the competitors in the final evaluation. We believe this apparent discrepancy arises because our approach explicitly computes approximate best responses at each iteration, rather than relying on a single fixed equilibrium policy. As a result, while the final meta-strategy may remain exploitable in theory, the growing population of strong counter-strategies allows our agent to adapt effectively to a wide range of opponents in practice.

To evaluate our training, the meta-game values and the best responses for each team were tracked over iterations. In particular, the meta-game value is given by

$$v = \sigma_{13}^{\intercal} A \sigma_{24}$$

and is the expected payoff of the restricted meta-game when both teams play their current meta strategies. If v is greater than 0, then Team 13 has an advantage and vice versa for v less than 0. A plot for the meta-game value for configuration (4) is given in Figure 3.

Early in training, the meta-game value exhibits large fluctuations and is predominantly negative, indicating that the initial restricted game is highly incomplete and favors Team 24. As new best-response strategies are added to both populations, the value shifts substantially, reflecting the rapid discovery of strategically relevant deviations. This behavior is expected in Team PSRO, where early iterations expand the strategy space aggressively rather than refining equilibrium mixtures.

As training continues, the meta-game value increases and stabilizes, converging toward a positive plateau around $v \approx 0.38$. This stabilization suggests that the populations have grown sufficiently rich that additional best responses no longer significantly change the equilibrium payoff of the restricted game. In other words, the meta-game has reached a locally stable equilibrium with respect to the available populations.

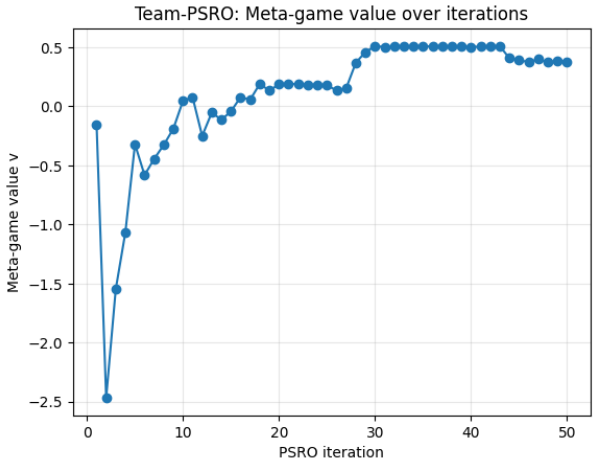


Fig. 3. Meta-game values over Team PSRO iterations for Configuration (4).

Importantly, this value should not be interpreted as the true game value of Leduc Hold'em, but rather as the equilibrium value of the restricted meta-game induced by the learned populations. The observed stabilization nonetheless indicates that Team-PSRO is successfully capturing a consistent strategic balance between the teams, even as the populations continue to grow.

Next, we analyze the best response gap values for configuration (4), which is given in Figure 4. The best-response gap curves measure the degree to which the current meta-strategies over the restricted populations can be exploited by newly computed oracle strategies. Specifically, the Team 13 gap corresponds to the improvement in payoff achievable by a best-responding Team 13 strategy against Team 24's meta-strategy, while the Team 24 gap measures the analogous quantity for Team 24. They are computed as follows:

$$\text{row gap} = \max_i (A\sigma_{24})_i - v, \quad \text{col gap} = v - \min_j (\sigma_{13}^{\intercal} A)_j$$

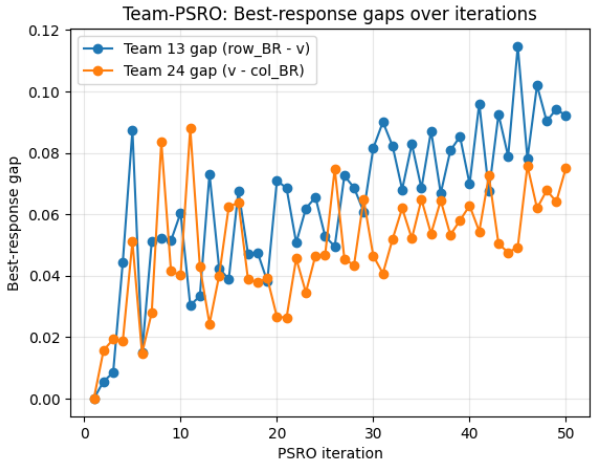


Fig. 4. Best response gap values over Team PSRO iterations for Configuration (4).

Across training, both gap curves remain strictly positive and exhibit noticeable oscillations rather than converging to zero. This behavior indicates that, despite stabilization of the meta-game value, the restricted meta-game has not converged to a full equilibrium in which no profitable deviations exist within the population space. Instead, each iteration continues to uncover new strategies that yield non-trivial improvements against the current meta-strategy.

Importantly, the persistence of non-zero gaps should not be interpreted as failure of the algorithm. In Team-PSRO, populations are intentionally expanded using approximate best-response oracles, and exploitability is measured with respect to an ever-changing strategy set. As the populations grow, the algorithm prioritizes discovering strategically relevant counter-strategies over strictly minimizing exploitability in the current restricted game. This naturally leads to sustained but bounded best-response gaps.

Moreover, the bounded nature of the gaps—remaining within a relatively narrow range rather than diverging—suggests that while new best responses continue to exist, their marginal benefit diminishes over time. This reflects a form of practical convergence: the meta-strategies become increasingly robust, even though exact equilibrium conditions are not achieved within the finite population and approximate oracle setting.

Overall, our Team PSRO implementation worked well for the competition.

VI. Reflection and Future Directions

Looking back at our project progress, we realized that we were initially deceived by the apparent simplicity of Team Leduc Hold'em. With only four players, six cards, and two betitng rounds, the game seemed amenable to intuive reasoning and hand-crafted strategies. We spent considerable time developing heuristic approaches up to the due date of Problem Set 3. Then we released that systematically designing strategies resistant to exploitation was far more difficult than anticipated.

After pivoting to algorithmic approaches, we invested significant effort implementing Team-PSRO before ultimately deciding to also explore Team Belief DAG as the challenge deadline approached. This sequential exploration consumed substantial time, leaving us with limited opportunity to deeply understand the literature related to Team Belief DAG. As a result, we were unable to develop creative extensions or optimizations tailored to the specific structure of our variant to Leduc Poker. We should have emphasized Team Belief DAG more as it was the solution concept for the competition.

In hindsight, committing earlier to a single principled approach and investing more time in validation and refinement would have yilded results than our broader but shallower exploration of multiple methods.

A. Future Directions for Team Belief DAG

Given additional time, we would pursue the following directions to improve our Team Belief DAG based implementation:

- Implementation Validation: Develop unit tests for individual components, connectivity grapy construction, public observation computation, prescription enuemration, and utility calculation. Testing these functions on games smaller than our current variant Poker would've been helpful.
- Extended Training: Run significantly more CFR+ iterations with more time to e eif our strategies eventually converge or remain stuck at a suboptimal plateau. While it's unlikely that our implementation is fully correct given the plateau, we never had the time to fully verify it.
- Diversity-Inducing Multi-Strategy Training: Rather than training multiple strategies independently from identical initializations, we could've explored methods to encourage diversity, such as adding explicit diversity regularization to the objective.
- Combining our Heuristic approach and see if that develops a better overall strategy: We could initialize CFR+ regrets based on our heuristic signaling strategies rather than uniform, potentially finding strategies that exploits opponents whose strategy is based on heuristics.

B. Future Directions for Team PSRO

The main limitation of our Team PSRO implementation is the computational overhead introduced by running both fictitious play at the meta-game level and CFR as a best-response oracle at every iteration. While both components are theoretically sound and well-suited for zero-sum extensive-form games, their repeated execution leads to suboptimal training times as the population sizes grow.

A natural direction for future work is to replace the CFR best-response oracle with function-approximation-based methods, such as multi-agent reinforcement learning (MARL). Approaches like MAPPO, as utilized in [2], amortize best-response computation across iterations and enable generalization across information sets, significantly reducing per-iteration cost in large games. Similarly, warm-starting CFR from previous regret tables or limiting CFR to a subset of strategically relevant information sets could further improve efficiency while preserving solution quality. Finally, variations of CFR could also be employed. For example, CFR+ accelerates convergence by truncating negative regrets while outcome-sampling CFR reduces per-iteration complexity through Monte-Carlo updates ([3], [4]).

On the meta-game side, fictitious play could be replaced with faster equilibrium solvers or alternative no-regret dynamics, such as multiplicative weights or regret matching. Finally, adaptive population management strategies, such as pruning dominated team strategies or merging similar policies, could help control population growth and promote scalability.

Together, these directions would allow Team PSRO to scale to substantially larger team-based imperfect-information games.

VII. Author Contributions

Andy was responsible to developing the heuristic strategy and implementing the Team Belief DAG based approach for this poker challenge. Specifically, Andy coded approximately 1600 lines of code handling game tree parsing, TB-DAG construction, CFR+ solving, and strategy extraction.

Christopher contributed to developing the heuristic strategy and implemented the Team PSRO approach for this poker challenge.

Acknowledgment

Despite not achieving the best results, we are still grateful for this poker challenge, as it provided us an opportunity to apply and solidify cocnepts form the course. Our work drew directly on the course's treatment of equilibrium concepts, regret minimization, and extensive-form games. We would like to thank Professor Farina and Professor Daskalakis for designing a fun course that is rooted in both theory and practice.

References

[1] Zhang, B. H., Farina, G., & Sandholm, T. (2022). Team Belief DAG: Generalizing the sequence form to team games for fast computation of correlated team Max-Min equilibria via regret minimization. arXiv (Cornell University). https://doi.org/10.48550/arxiv.2202.00789

[2] McAleer, S. M., Farina, G., Zhou, G., Wang, M., Yang, Y., & Sandholm, T. (2023). Team-PSRO for learning approximate TMECor in large team games via cooperative reinforcement learning. In Advances in Neural Information Processing Systems 36 (NeurIPS 2023).

[3] Tammelin, O. (2014). Solving Large Imperfect Information Games Using CFR+. arXiv. https://doi.org/10.48550/arXiv.1407.5042

[4] Lanctot, M., Waugh, K., Zinkevich, M., & Bowling, M. (2009). Monte Carlo Sampling for Regret Minimization in Extensive Games. In Advances in Neural Information Processing Systems 22 (NeurIPS 2009).